

Introduction to Large Language Models Applications

Final Exam

May 29, 2026

Instructions.

1. The exam contains 6 problems. You will have 90 minutes to complete the exam.
 2. Partial credit will be awarded, so please show your reasoning process clearly.
 3. Please write your answers using only English, mathematical notation, and diagrams.
 4. The exam's total score is 100 points plus 5 bonus points. We recommend that you spend time on bonus problems only after you have completed all regular problems.
-

Problem 1. (16 points) (KL Divergence.)

The Kullback–Leibler (KL) divergence is a quantity frequently used in deep learning. For two discrete probability distributions p and q over a common vocabulary $\mathcal{V}$, it is defined as

$$\text{KL}(p \parallel q) = \sum_{v \in \mathcal{V}} p(v) \log \frac{p(v)}{q(v)}.$$

- (1) (8 points) Prove that $\text{KL}(p \parallel q) \geq 0$ for all probability distributions p and q , with equality if and only if $p = q$. For simplicity, you can assume $p(x) > 0$ and $q(x) > 0$ for all x .
- (2) (8 points) Consider the following training objective. Let p_{data} denote the (fixed) data distribution and p_{θ} the model distribution. Show that minimizing the cross-entropy

$$-\sum_{v \in \mathcal{V}} p_{\text{data}}(v) \log p_{\theta}(v)$$

with respect to θ is equivalent to minimizing $\text{KL}(p_{\text{data}} \parallel p_{\theta})$.

Hint. A formal derivation is required for both questions.

Solution 1.

- (1) Since $\log$ is concave, Jensen's inequality with weights $p(v)$ ($\sum_v p(v) = 1$) and points $q(v)/p(v)$ gives

$$-\text{KL}(p \parallel q) = \sum_{v \in \mathcal{V}} p(v) \log \frac{q(v)}{p(v)} \leq \log \left(\sum_{v \in \mathcal{V}} p(v) \cdot \frac{q(v)}{p(v)} \right) = \log \left(\sum_{v \in \mathcal{V}} q(v) \right) = \log 1 = 0,$$

hence $\text{KL}(p \parallel q) \geq 0$.

Since $\log$ is *strictly* concave, equality holds iff all points $q(v)/p(v)$ are equal to a common constant c . Summing $q(v) = cp(v)$ over v gives $c = \sum_v q(v) = 1$, so $q(v) = p(v)$ for all v , i.e. $p = q$. Conversely $p = q$ gives $\log 1 = 0$ in every term, so $\text{KL} = 0$.

- (2) Expand the KL divergence between the true conditional distribution p_{data} and the model's predicted distribution p_{θ} :

$$\text{KL}(p_{\text{data}} \parallel p_{\theta}) = \sum_{v \in V} p_{\text{data}}(v) \log \frac{p_{\text{data}}(v)}{p_{\theta}(v)} = \sum_{v \in V} p_{\text{data}}(v) \log p_{\text{data}}(v) - \sum_{v \in V} p_{\text{data}}(v) \log p_{\theta}(v).$$

Identify the two terms as the negative entropy of p_{data} and the cross-entropy:

$$\sum_{v \in V} p_{\text{data}}(v) \log p_{\text{data}}(v) = -H(p_{\text{data}}), \quad - \sum_{v \in V} p_{\text{data}}(v) \log p_{\theta}(v) = H(p_{\text{data}}, p_{\theta}),$$

so that

$$\text{KL}(p_{\text{data}} \parallel p_{\theta}) = \underbrace{-H(p_{\text{data}})}_{\text{independent of } \theta} + \underbrace{H(p_{\text{data}}, p_{\theta})}_{\text{cross-entropy}}.$$

The term $-H(p_{\text{data}})$ depends only on the true distribution and is therefore *constant with respect to* θ . Hence

$$\text{KL}(p_{\text{data}} \parallel p_{\theta}) = \left(- \sum_{v \in V} p_{\text{data}}(v) \log p_{\theta}(v) \right) + \text{const.}$$

The cross-entropy and the KL divergence differ only by a θ -independent constant, so they have the same minimizer and the same gradient in θ :

$$\arg \min_{\theta} \left(- \sum_{v \in V} p_{\text{data}}(v) \log p_{\theta}(v) \right) = \arg \min_{\theta} \text{KL}(p_{\text{data}} \parallel p_{\theta}).$$

That is, minimizing the cross-entropy with respect to θ is equivalent to minimizing $\text{KL}(p_{\text{data}} \parallel p_{\theta})$.

Remark. It is acceptable to treat p_{data} as one-hot on the actual next token v^* ; then $H(p_{\text{data}}) = 0$.

Problem 2. (12 points) (LoRA Diagram and Latency Analysis.)

Instead of full fine-tuning, you decide to use LoRA to adapt a 7B parameter model to a specific task.

- (1) (8 points) Draw a diagram representing a single weight matrix $W \in \mathbb{R}^{d \times d}$ being adapted by LoRA. Your drawing **must** include:

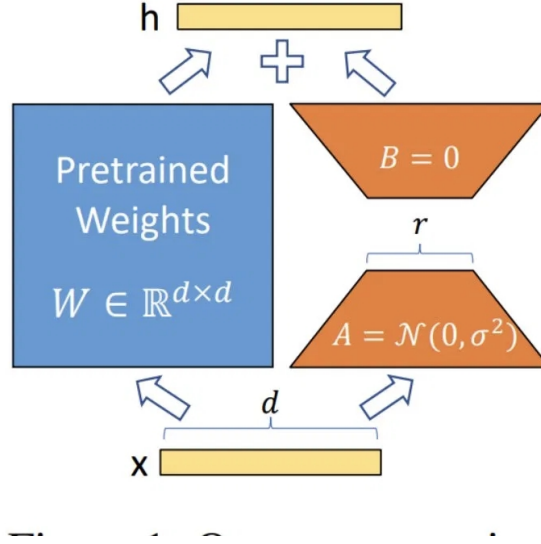
- The Pre-trained (frozen) weights W .
- The two low-rank matrices (A and B) with their respective ranks and dimensions (r).
- The input x and how it flows through both the frozen and trainable paths to produce the output hidden state h .
- Clearly indicate which parts are updated during back-propagation and which are frozen.

- (2) (4 points) Briefly explain (in natural language) why LoRA introduces zero latency during inference.

Solution 2.

- (1) **Diagram.**

We reproduce the canonical LoRA diagram (Hu et al., 2021):



Roles:

- $W \in \mathbb{R}^{d \times d}$ (blue): pre-trained weights, **frozen** (no gradient).
- $A \in \mathbb{R}^{r \times d}$ (orange, bottom): down-projection, **trainable**, initialized as Gaussian $\mathcal{N}(0, \sigma^2)$.
- $B \in \mathbb{R}^{d \times r}$ (orange, top): up-projection, **trainable**, initialized to zero. This ensures $h = Wx$ at the start, so adaptation begins from the pre-trained model.
- $r \ll d$: low-rank bottleneck, the source of parameter efficiency.

Forward pass: input x flows through both paths in parallel: Wx via the frozen path and BAx via the trainable path. They are summed to produce $h = Wx + BAx$.

Backward pass: gradients flow through both paths, but only A and B receive parameter updates; W remains frozen.

(2) Zero-Latency at Inference.

The output of the LoRA-adapted layer is, by the linearity of matrix multiplication,

$$h = Wx + BAx = (W + BA)x.$$

The matrix $BA \in \mathbb{R}^{d \times d}$ has *exactly the same shape* as W . Therefore, at deployment time, one can pre-compute once and for all:

$$W' := W + BA \in \mathbb{R}^{d \times d},$$

and store W' as a *single* weight matrix. From that point on, inference performs only:

$$h = W'x,$$

which is one matrix-vector multiplication — **identical in cost to the original pre-trained layer**. No extra computation, no extra memory access, no extra latency.

Contrast with sequential adapters. A traditional adapter inserts a small MLP *in series* on the residual path: $h = Wx + \text{Adapter}(Wx)$. The adapter must be executed sequentially after W , contributing additional FLOPs and latency at every forward pass. LoRA avoids this because the trainable branch BAx runs in *parallel* to Wx , and the linearity of the sum allows the two matrices to be merged.

Problem 3. (20 points) **(5 bonus points)** (Uniqueness of RoPE as a Relative Position Encoding.)

Motivation. RoPE is usually *defined* by 2D block-diagonal rotations and then *shown* to satisfy a relative-position property. This problem goes the other way: assuming only the relative-position property, let's prove that RoPE is unique (up to an orthogonal transform). However, the proof is quite long, and we will only work out some initial steps.

Setup. Let d be even (the standard case in Transformers). Let $f : \mathbb{R} \rightarrow \mathbb{R}^{d \times d}$ be a C^1 matrix-valued function* with $f(0) = I$. We say f has the *relative-position property* iff there exists a function g such that for all $\vec{q}, \vec{k} \in \mathbb{R}^d$ and all $m, n \in \mathbb{R}$,

$$\langle f(m)\vec{q}, f(n)\vec{k} \rangle = g(n - m; \vec{q}, \vec{k}).$$

- (1) (8 points) Prove that $f(m)^\top f(n) = f(n - m)$ as a matrix identity for all $m, n \in \mathbb{R}$.
- (2) (3 points) Show that $f(m)$ is an orthogonal matrix for every m , i.e., $f(m)^\top f(m) = I$. You can use the result of (1).
- (3) (3 points) Prove the one-parameter group law: $f(m + n) = f(m)f(n)$.
- (4) (6 points) Let $A := f'(0) \in \mathbb{R}^{d \times d}$. By differentiating the identity in (3), show

$$f'(m) = f(m)A.$$

- (5) **(2 bonus points)** Show that the matrix A in (4) must be skew-symmetric, i.e.,

$$A^\top + A = 0.$$

Hint. Differentiate (2) at $m = 0$.

- (6) **(3 bonus points)** *You may use without proof:*

- Any skew-symmetric $A \in \mathbb{R}^{d \times d}$ (with d even) admits an orthogonal matrix U such that

$$U^\top A U = \text{blockdiag}\left(\begin{pmatrix} 0 & -\theta_1 \\ \theta_1 & 0 \end{pmatrix}, \dots, \begin{pmatrix} 0 & -\theta_{d/2} \\ \theta_{d/2} & 0 \end{pmatrix}\right).$$

- The ODE in (4) with $f(0) = I$ has the unique solution $f(m) = \exp(mA)$, and $\exp(U^\top A U \cdot m) = U^\top \exp(mA) U$.
- For any $\theta \in \mathbb{R}$, $\exp\left(m \begin{pmatrix} 0 & -\theta \\ \theta & 0 \end{pmatrix}\right) = \begin{pmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{pmatrix} =: R(m\theta)$.

Combine these to prove: there exists an orthogonal matrix U such that

$$U^\top f(m) U = \text{blockdiag}(R(m\theta_1), \dots, R(m\theta_{d/2})).$$

This is RoPE.

Solution 3.

- (1) **Prove** $f(m)^\top f(n) = f(n - m)$.

For all $\vec{q}, \vec{k} \in \mathbb{R}^d$ and all $m, n \in \mathbb{R}$:

$$\langle f(m)\vec{q}, f(n)\vec{k} \rangle = g(n - m; \vec{q}, \vec{k}).$$

*A function with continuous first-order derivative.

The left side can be written as

$$\langle f(m)\vec{q}, f(n)\vec{k} \rangle = (f(m)\vec{q})^\top (f(n)\vec{k}).$$

Rearranging,

$$\vec{q}^\top f(m)^\top f(n) \vec{k} = g(n - m; \vec{q}, \vec{k}).$$

Define

$$M(n, m) = f(m)^\top f(n).$$

Take any n', m' . If $n - m = n' - m'$, then for any pair $\vec{q}, \vec{k}$ we have

$$\vec{q}^\top M(n, m) \vec{k} = g(n - m; \vec{q}, \vec{k}) = g(n' - m'; \vec{q}, \vec{k}) = \vec{q}^\top M(n', m') \vec{k}.$$

By the arbitrariness of $\vec{q}, \vec{k}$ (take the standard basis vectors e_i, e_j), we get $M(n, m) = M(n', m')$. Hence $M(n, m)$ depends only on $n - m$, and there exists f' such that $M(n, m) = f'(n - m)$.

Next, we show $f' = f$. We already have

$$f(m)^\top f(n) = f'(n - m) \quad \text{for all } m, n \in \mathbb{R}.$$

Setting $m = 0$ and using $f(0) = I$:

$$f'(n) = f(0)^\top f(n) = I^\top f(n) = f(n) \quad \text{for all } n \in \mathbb{R}.$$

This directly gives $f' = f$. Substituting $f' = f$ yields the desired matrix identity:

$$\boxed{f(m)^\top f(n) = f(n - m) \quad \text{for all } m, n \in \mathbb{R}.}$$

(2) **$f(m)$ is orthogonal.**

Set $n = m$ in the identity just proved:

$$f(m)^\top f(m) = f(m - m) = f(0) = I.$$

This is the definition of an orthogonal matrix. □

(3) **One-parameter group law:** $f(m + n) = f(m)f(n)$.

Start from (1) and replace n by $m + n$:

$$f(m)^\top f(m + n) = f((m + n) - m) = f(n).$$

Now left-multiply both sides by $f(m)$:

$$f(m) f(m)^\top f(m + n) = f(m) f(n).$$

By (2), $f(m)$ is orthogonal, so $f(m) f(m)^\top = I$. The left side becomes $f(m + n)$:

$$\boxed{f(m + n) = f(m) f(n).}$$

(4) **Differentiating the group law.**

Fix m and differentiate $f(m+n) = f(m)f(n)$ with respect to n . The left side is

$$\frac{d}{dn}f(m+n) = f'(m+n),$$

by the chain rule. The right side, since $f(m)$ does not depend on n , is

$$\frac{d}{dn}[f(m)f(n)] = f(m)f'(n).$$

Therefore for all m, n :

$$f'(m+n) = f(m)f'(n).$$

Set $n = 0$ and recall $A := f'(0)$:

$$\boxed{f'(m) = f(m)A.}$$

(5) **Differentiating (2)**

By (2), $f(m)^\top f(m) = I$ identically in m . Differentiate both sides with respect to m . The right side gives 0. For the left side, apply the product rule:

$$\frac{d}{dm}[f(m)^\top f(m)] = (f'(m))^\top f(m) + f(m)^\top f'(m) = 0.$$

Substitute $f'(m) = f(m)A$ from Step 1:

$$\begin{aligned} (f(m)A)^\top f(m) + f(m)^\top (f(m)A) &= 0, \\ A^\top \underbrace{f(m)^\top f(m)}_{=I} + \underbrace{f(m)^\top f(m)}_{=I} A &= 0, \\ \boxed{A^\top + A} &= 0. \end{aligned}$$

Hence A is skew-symmetric. □

(6) **Block-diagonal RoPE form**

By the given facts:

- Since A is skew-symmetric (from (4)), there exists an orthogonal U and real numbers $\theta_1, \dots, \theta_{d/2}$ such that

$$U^\top A U = \Lambda := \text{blockdiag}\left(\begin{pmatrix} 0 & -\theta_1 \\ \theta_1 & 0 \end{pmatrix}, \dots, \begin{pmatrix} 0 & -\theta_{d/2} \\ \theta_{d/2} & 0 \end{pmatrix}\right).$$

- Since $f'(m) = f(m)A$ with $f(0) = I$, the unique solution is $f(m) = \exp(mA)$.

Step 1: rewrite $f(m)$ via U . Since U is orthogonal, $UU^\top = I$, so

$$A = U(U^\top A U)U^\top = U\Lambda U^\top,$$

and therefore

$$mA = U(m\Lambda)U^\top.$$

By the given fact $\exp(U^\top XU) = U^\top \exp(X)U$ (applied with $X = mA$), equivalently $\exp(UXU^\top) = U \exp(X)U^\top$:

$$f(m) = \exp(mA) = \exp(U(m\Lambda)U^\top) = U \exp(m\Lambda)U^\top.$$

Rearranging:

$$U^\top f(m) U = \exp(m\Lambda).$$

Step 2: compute $\exp(m\Lambda)$. Since $m\Lambda$ is block-diagonal, its exponential acts block-by-block. By the third given fact, each 2×2 block satisfies

$$\exp\left(m \begin{pmatrix} 0 & -\theta_j \\ \theta_j & 0 \end{pmatrix}\right) = \begin{pmatrix} \cos(m\theta_j) & -\sin(m\theta_j) \\ \sin(m\theta_j) & \cos(m\theta_j) \end{pmatrix} = R(m\theta_j).$$

Therefore

$$\exp(m\Lambda) = \text{blockdiag}(R(m\theta_1), \dots, R(m\theta_{d/2})).$$

Combining the two steps:

$$U^\top f(m) U = \text{blockdiag}(R(m\theta_1), \dots, R(m\theta_{d/2})).$$

This is RoPE in block-diagonal form. □

Problem 4. (15 points) (Variance Reduction in Policy Gradient.)

Setup. Let $\pi_\theta(y)$ be a parameterized distribution over a finite action set $\mathcal{Y}$, with $\theta \in \mathbb{R}^d$. Assume $\pi_\theta(y) > 0$ and is differentiable in θ for every y . Let $r : \mathcal{Y} \rightarrow \mathbb{R}$ be a reward (*independent of θ*). The objective is

$$J(\theta) := \mathbb{E}_{y \sim \pi_\theta}[r(y)].$$

Define the *score function*

$$g_\theta(y) := \nabla_\theta \log \pi_\theta(y) \in \mathbb{R}^d.$$

(1) (8 points) [*Policy gradient theorem.*] Prove

$$\nabla_\theta J(\theta) = \mathbb{E}_{y \sim \pi_\theta}[r(y) g_\theta(y)].$$

Your derivation must explicitly use the identity $\nabla_\theta \pi_\theta(y) = \pi_\theta(y) g_\theta(y)$.

(2) (4 points) [*Score has zero mean.*] Prove $\mathbb{E}_{y \sim \pi_\theta}[g_\theta(y)] = 0$. *Hint:* differentiate the identity $\sum_y \pi_\theta(y) = 1$.

(3) (3 points) [*Baseline preserves the gradient.*] For any scalar $b \in \mathbb{R}$ depending only on θ (not on y), prove

$$\nabla_\theta J(\theta) = \mathbb{E}_{y \sim \pi_\theta}[(r(y) - b) g_\theta(y)].$$

(When doing RL training, we can reduce the variance of the estimated loss by choosing an appropriate b . However, this is out of the scope of this exam.)

Solution 4.

(1) **Policy gradient theorem.**

We want to compute $\nabla_\theta J(\theta)$ where

$$J(\theta) = \mathbb{E}_{y \sim \pi_\theta}[r(y)] = \sum_{y \in \mathcal{Y}} \pi_\theta(y) r(y).$$

The reward $r(y)$ does not depend on θ . Since $\mathcal{Y}$ is finite, we can interchange the gradient and the sum:

$$\nabla_\theta J(\theta) = \nabla_\theta \sum_y \pi_\theta(y) r(y) = \sum_y r(y) \nabla_\theta \pi_\theta(y).$$

The key identity. For any positive function $\pi_\theta(y) > 0$,

$$\nabla_\theta \log \pi_\theta(y) = \frac{\nabla_\theta \pi_\theta(y)}{\pi_\theta(y)},$$

by the chain rule applied to log. Rearranging:

$$\nabla_\theta \pi_\theta(y) = \pi_\theta(y) \nabla_\theta \log \pi_\theta(y) = \pi_\theta(y) g_\theta(y).$$

This is the *log-derivative trick* (or REINFORCE trick).

Substituting back:

$$\nabla_\theta J(\theta) = \sum_y r(y) \pi_\theta(y) g_\theta(y) = \mathbb{E}_{y \sim \pi_\theta} [r(y) g_\theta(y)]. \quad \square$$

(2) **Score has zero mean.**

Differentiate the identity $\sum_y \pi_\theta(y) = 1$ with respect to θ . The right side is constant, so:

$$\nabla_\theta \sum_y \pi_\theta(y) = \sum_y \nabla_\theta \pi_\theta(y) = 0.$$

Using the same log-derivative trick $\nabla_\theta \pi_\theta(y) = \pi_\theta(y) g_\theta(y)$:

$$\sum_y \pi_\theta(y) g_\theta(y) = \mathbb{E}_{y \sim \pi_\theta} [g_\theta(y)] = 0. \quad \square$$

(3) **Baseline preserves the gradient.**

Using linearity of expectation and result (2):

$$\mathbb{E}_{y \sim \pi_\theta} [(r(y) - b) g_\theta(y)] = \mathbb{E}[r(y) g_\theta(y)] - b \mathbb{E}[g_\theta(y)].$$

The first term equals $\nabla_\theta J$ by (1). The second term: since b does not depend on y , it can be pulled outside the expectation, and $\mathbb{E}[g_\theta(y)] = 0$ by (2). Therefore:

$$\mathbb{E}_{y \sim \pi_\theta} [(r(y) - b) g_\theta(y)] = \nabla_\theta J(\theta) - b \cdot 0 = \nabla_\theta J(\theta). \quad \square$$

Problem 5. (10 points) (Tensor Parallel for MLP Layer.)

Describe how tensor parallelism is implemented on the forward pass of a typical MLP layer. Specifically, consider the following simplified version of a two-layer MLP layer: given input matrix $X \in \mathbb{R}^{n \times h_1}$ and weight matrices $A \in \mathbb{R}^{h_1 \times h_2}$, $B \in \mathbb{R}^{h_2 \times h_3}$, we compute

$$\begin{aligned} Y &= u_1(XA), \\ Z &= u_2(YB), \end{aligned}$$

where u_1, u_2 are fixed activation functions applied element-wise. The matrix Z is considered as the final output of the layer.

Now, assume that a total of k GPUs take part in the computation. The i -th GPU processes a $1/k$ fraction of all weights $A_i \in \mathbb{R}^{h_1 \times (h_2/k)}$, $B_i \in \mathbb{R}^{(h_2/k) \times h_3}$. We may assume

$$A = [A_1, A_2, \dots, A_k], B = \begin{bmatrix} B_1 \\ B_2 \\ \dots \\ B_k \end{bmatrix}.$$

Assume that at the start of the computation, each GPU processes an identical copy of X , and the goal is that at the end of the computation, each GPU processes an identical copy of Z .

Your task is to demonstrate a way to complete this computation task that exploits full parallelism. You should clearly state what should be computed on each GPU for each step of computation, and how should they communicate. You are free to use formulas and draw diagrams to clarify your explanation.

Hint. In your answer, you may directly use the term *all-reduce* which does the following: an all-reduce on distributively stored matrices $M_1, \dots, M_k$ computes $M = M_1 + \dots + M_k$ and replace all of $M_1 \sim M_k$ by M in place. This operation requires that the matrices all have the same shape. You may assume that the optimal implementation of this operation is well-studied and you do not have to explain it in your answer.

Solution 5. For simplicity, set $T = YB$. GPU i first computes

$$\begin{aligned} Y_i &= u_1(XA_i), \\ T_i &= Y_i B_i \end{aligned}$$

and all GPUs perform an all-reduce on T_i -s to obtain T , which is equal to YB in the single GPU scenario. Each GPU then applies u_2 elementwise on the copy of T they process.

Remark. There is another solution with less compute cost: reduce-scatter on the T_i -s, each GPU applies u_2 on its $1/k$ fraction of T to get Z , and perform an all-gather to obtain the final Z . Both solutions receive full credits.

Problem 6. (27 points) (Mixture-of-Experts Implementation of DeepSeek-V3.)

Read the following excerpt from the official DeepSeek-V3 model definition. This excerpt implements the Mixture-of-Experts (MoE) gating module of DeepSeek-V3.

```

1  class MoEGate(nn.Module):
2      def __init__(self, config):
3          super().__init__()
4          self.config = config
5          self.top_k = config.num_experts_per_tok
6          self.n_routed_experts = config.n_routed_experts
7          self.routed_scaling_factor = config.routed_scaling_factor
8          self.scoring_func = config.scoring_func
9          self.topk_method = config.topk_method
10         self.n_group = config.n_group
11         self.topk_group = config.topk_group
12
13         # topk selection algorithm
14         self.norm_topk_prob = config.norm_topk_prob
15         self.gating_dim = config.hidden_size
16         self.weight = nn.Parameter(
17             torch.empty((self.n_routed_experts, self.gating_dim))
18         )
19         if self.topk_method == "noaux_tc":
20             self.e_score_correction_bias = nn.Parameter(
21                 torch.empty((self.n_routed_experts))
22             )
23         self.reset_parameters()
24
25     def reset_parameters(self) -> None:
26         import torch.nn.init as init
27
28         init.kaiming_uniform_(self.weight, a=math.sqrt(5))

```

```

29
30 def forward(self, hidden_states):
31     bsz, seq_len, h = hidden_states.shape
32     ### compute gating score
33     hidden_states = hidden_states.view(-1, h)
34     logits = F.linear(
35         hidden_states.type(torch.float32), self.weight.type(torch.float32), None
36     )
37     if self.scoring_func == "sigmoid":
38         scores = logits.sigmoid()
39     else:
40         raise NotImplementedError(
41             f"insupportable scoring function for MoE gating: {self.scoring_func}"
42         )
43
44     ### select top-k experts
45     if self.topk_method == "noaux_tc":
46         assert not self.training
47         scores_for_choice = scores.view(bsz * seq_len, -1) +
48             ↪ self.e_score_correction_bias.unsqueeze(0)
49         group_scores = (
50             scores_for_choice.view(bsz * seq_len, self.n_group, -1).topk(2,
51             ↪ dim=-1)[0].sum(dim = -1)
52         ) # [n, n_group]
53         group_idx = torch.topk(
54             group_scores, k=self.topk_group, dim=-1, sorted=False
55         )
56         [
57             1
58         ] # [n, top_k_group]
59         group_mask = torch.zeros_like(group_scores) # [n, n_group]
60         group_mask.scatter_(1, group_idx, 1) # [n, n_group]
61         score_mask = (
62             group_mask.unsqueeze(-1)
63             .expand(
64                 bsz * seq_len, self.n_group, self.n_routed_experts // self.n_group
65             )
66             .reshape(bsz * seq_len, -1)
67         ) # [n, e]
68         tmp_scores = scores_for_choice.masked_fill(~score_mask.bool(), float("-inf")) # [n,
69             ↪ e]
70         _, topk_idx = torch.topk(
71             tmp_scores, k=self.top_k, dim=-1, sorted=False
72         )
73         topk_weight = scores.gather(1, topk_idx)
74     else:
75         raise NotImplementedError(
76             f"insupportable TopK function for MoE gating: {self.topk_method}"
77         )
78
79     ### norm gate to sum 1
80     if self.top_k > 1 and self.norm_topk_prob:
81         denominator = topk_weight.sum(dim=-1, keepdim=True) + 1e-20
82         topk_weight = topk_weight / denominator
83         topk_weight = topk_weight * self.routed_scaling_factor # must multiply the scaling factor
84
85     return topk_idx, topk_weight

```

- (1) (12 points) Write down the shape of the following tensors.

<code>logits</code>	(Line 34)
<code>e_score_correction_bias</code>	(Line 47)
<code>group_scores</code>	(Line 48)
<code>score_mask</code>	(Line 58)

Hint. We recommend that you solve this task in parallel with the two tasks below.

Hint. You may answer with the terms: `bsz`, `seq_len`, `h` (we assume this matches `self.gating_dim` so you may use either), `self.top_k`, `self.n_routed_experts`, `self.n_group` and `self.topk_group`.

- (2) (3 points) On Line 47, an additional term `e_score_correction_bias` is added to the scores. What is this term used for?

Hint. By DeepSeek-V3 technical report, this term is tuned during training, and fixed during inference.

Hint. The standard answer to this question consists of only two English words, which happens to be exactly the title of one of our lecture slides! However, you are free to answer with any number of words.

- (3) (12 points) In this code, a mysterious method called `noaux_tc` (as Line 19 and Line 44 show) is applied to select Top- k experts. Explain how this method works by analyzing Line 48-68.

Hint. The name itself (shortened form of *No Auxiliary-loss Token-Choice*) carries not much information; you should focus solely on the code.

Hint. Don't be scared by the amount of code; analyze them line by line and finally you will understand the algorithm as a whole. Partial scores will be awarded in proportion to the amount of code that you have correctly analyzed.

Hint. You may find the following PyTorch documentation useful:

- `torch.topk(input, k, dim, sorted=False)` returns the largest k elements of `input` along dimension `dim`. It returns a pair (`values`, `indices`). For example, if

$$x = [0.1, 0.7, 0.3, 0.9],$$

then `torch.topk(x, k=2)` returns values `[0.9, 0.7]` and indices `[3, 1]`.

- `Tensor.scatter_(dim, index, value)` writes `value` into a tensor at positions specified by `index` along dimension `dim`. For example, if

$$\text{group_mask} = [0, 0, 0, 0]$$

and the selected group indices are

$$\text{group_idx} = [1, 3],$$

then after `group_mask.scatter_(0, group_idx, 1)`, the mask becomes

$$[0, 1, 0, 1].$$

- `Tensor.expand(shape)` expands a tensor along dimensions of size 1 without copying data. The length of the given list `shape` must be the same as the number of dimensions of the tensor.
- `Tensor.masked_fill(mask, value)` replaces all entries whose corresponding mask value is `True` by `value`.

Acknowledgement. The code is adopted from official repository of DeepSeek-V3 on Hugging Face.

Solution 6.

(1)

```
logits          [bsz, seq_len, self.n_routed_experts]
e_score_correction_bias [self.n_routed_experts]
group_scores     [bsz * seq_len, self.n_group]
score_mask       [bsz * seq_len, self.n_routed_experts]
```

(2) Load balancing.

(3) The algorithm divides experts into expert groups, computes the sum of top-2 expert scores in each group (Line 48-50), and selects the groups with sum ranking top `self.topk_group` among all groups (Line 51-55). Only experts in these groups are allowed (through masking, Line 56-65). The algorithm then selects the top `self.top_k` experts from these groups (Line 66-68).